

SVX Library Class Catalog

SystemVerilog Extension Library — Core Library (**src/**)

Version 2.1.2 beta

Mark Glasser

September 2026

Contents

1	Types	3
1.1	svx_types — Standard Typedefs	3
1.2	type_handle	3
1.3	type_match	4
1.4	typeid	5
2	Traits	7
2.1	Traits Classes	7
3	Containers	10
3.1	object — Base Object	10
3.2	container — Container Base Classes	10
3.3	vector	11
3.4	map	12
3.5	queue	13
3.6	stack	14
3.7	deque	14
3.8	circ_buf — Circular Buffer	15
3.9	pair / triple / quadruple	16
3.10	sorter	16
4	Iterators	18
4.1	Iterator Interfaces	18
4.2	List Iterators	19
4.3	Map Iterators	21
4.4	Permutation Iterators	22
4.5	range	24
4.6	map_range	25
5	Algorithms	26
5.1	algo — Core Algorithms	26
5.2	algo2 — Dual-Iterator Algorithms	27
5.3	accum — Accumulation Algorithms	27
5.4	fcn — Function Objects	28
5.5	predicate — Predicate Classes	29
6	Linked Structures	31
6.1	node	31
6.2	tree	31
6.3	tree_iterator	32

7	Set	34
7.1	set	34
8	Utilities	35
8.1	msg — Message Interface	35
8.2	msg macros — Convenience Message Macros	35
8.3	ver — Version	36

Types

1.1 svx_types — Standard Typedefs

File: src/types/svx_types.svh

Standardized C-style integer typedefs, size and index types, process identifiers, and package-level Boolean constants. Provides a portable numeric vocabulary across the library.

Typedef	Base Type	Description
int8_t	byte	Signed 8-bit integer.
uint8_t	byte unsigned	Unsigned 8-bit integer.
int16_t	shortint	Signed 16-bit integer.
uint16_t	shortint unsigned	Unsigned 16-bit integer.
int32_t	int	Signed 32-bit integer.
uint32_t	int unsigned	Unsigned 32-bit integer.
int64_t	longint	Signed 64-bit integer.
uint64_t	longint unsigned	Unsigned 64-bit integer.
int128_t	bit [127:0]	Signed 128-bit integer (two-state).
uint128_t	bit unsigned [127:0]	Unsigned 128-bit integer (two-state).
size_t	uint64_t	Represents a container or array size.
index_t	uint64_t	Unsigned container index.
signed_index_t	int64_t	Signed container index, used for skip distances and offsets.
pid_t	int unsigned	Process identifier.

1.2 type_handle

File: src/types/type_handle.svh

Singleton type-handle mechanism. `type_handle#(T)` is a parameterized class that returns a unique singleton object for each distinct type T, enabling runtime type identity comparison. `type_container#(T)` wraps any value with its type handle for polymorphic storage.

`type_handle_base`

virtual class `extends` object

Non-parameterized base for all type handles.

Method	Return	Args	Description
<code>get_type_handle</code>	<code>type_handle_base</code>	—	Return the singleton type handle for this object (pure virtual).

`type_handle#(T)`

class **extends** `type_handle_base`

Parameterized singleton. One instance exists per distinct type T.

Method	Return	Args	Description
<code>get_type</code>	<code>type_handle#()</code>	—	Return the singleton type handle for T, creating it on the first call. Static.
<code>get_type_handle</code>	<code>type_handle_base</code>	—	Return this singleton cast to base type.

`type_container_base`

class

Non-parameterized base for typed containers; supports polymorphic storage with `convert2string`.

Method	Return	Args	Description
<code>convert2string</code>	<code>string</code>	—	Return a human-readable representation (default: empty string).
<code>get_type_handle</code>	<code>type_handle_base</code>	—	Return the type handle for the contained type (default: null).

`type_container#(T)`

class **extends** `type_container_base`

Holds a value of type T along with its type handle.

Constructor: `new(--)` — Initialise the type-handle singleton for T.

Method	Return	Args	Description
<code>get_type_handle</code>	<code>type_handle_base</code>	—	Return the type handle for T.
<code>set</code>	<code>void</code>	T x	Store a new value, replacing any existing one.
<code>get</code>	T	—	Retrieve the stored value.

1.3 `type_match`

File: `src/types/type_match.svh`

Compile-time type-compatibility utilities. `type_match#(T1,T2)` provides static functions to check whether two type parameters are identical or whether one is derived from the other. The convenience macro ``check_is_derived_from` installs a const-initializer that will call `$fatal` if the constraint is violated at elaboration time.

`type_match#(T1, T2)`

class

Static type-comparison utilities for T1 and T2.

Method	Return	Args	Description
<code>test_is_match</code>	<code>bit</code>	—	True if T1 and T2 are the same type. Static.
<code>is_match</code>	<code>bit</code>	<code>int line,</code> <code>string file</code>	True if match; fatal error with location if not. Static.
<code>test_is_derived_from</code>	<code>bit</code>	—	True if T1 is derived from T2 (via <code>\$cast</code> probe). Static.
<code>is_derived_from</code>	<code>bit</code>	<code>int line,</code> <code>string file</code>	True if derived; fatal error if not. Static.
<code>fail_match</code>	<code>void</code>	<code>int line,</code> <code>string file</code>	Issue <code>\$fatal</code> reporting type mismatch. Static.
<code>fail_derived</code>	<code>void</code>	<code>int line,</code> <code>string file</code>	Issue <code>\$fatal</code> reporting derivation failure. Static.

1.4 typeid

File: src/types/typeid.svh

Runtime type-classification for common SystemVerilog scalar types. `typeid#(T)` uses `type_handle` singletons to determine at run time whether T is integral, two-state, four-state, real, or string. Classification is limited to explicitly enumerated built-in types.

`typeid#(T)`

class

Static type-classification functions for built-in SV scalar types.

Method	Return	Args	Description
<code>is_int</code>	<code>bit</code>	—	True if T is any integral type. Static.
<code>is_int_fail</code>	<code>bit</code>	—	Same as <code>is_int()</code> ; prints error and calls <code>\$finish</code> if false. Static.
<code>is_two_state</code>	<code>bit</code>	—	True if T is integral and two-state (e.g., <code>bit</code> , <code>int</code>). Static.
<code>is_two_state_fail</code>	<code>bit</code>	—	Same but terminates on failure. Static.

Method	Return	Args	Description
<code>is_four_state</code>	bit	—	True if T is logic, reg, integer, or time. Static.
<code>is_four_state_fail</code>	bit	—	Same but terminates on failure. Static.
<code>is_real</code>	bit	—	True if T is real. Static.
<code>is_real_fail</code>	bit	—	Same but terminates on failure. Static.
<code>is_string</code>	bit	—	True if T is string. Static.
<code>is_string_fail</code>	bit	—	Same but terminates on failure. Static.

Traits

2.1 Traits Classes

File: src/containers/traits.svh

The traits system provides per-type constants and static functions used by all parameterized containers and algorithms. Every traits class defines an `empty_t` typedef, an `empty` constant, and static `equal()`, `compare()`, and `sort()` functions. The base class is `void_traits`; scalar integer families derive from `base_int_traits`.

`void_traits`

class `extends` `void_t`

Base traits class. `empty` is null (`void_t`). All type-flag localparams default to false.

Method	Return	Args	Description
<code>equal</code>	<code>bit</code>	<code>void_t a, b</code>	Always returns 1 (void objects are equivalent). Static.
<code>compare</code>	<code>int32_t</code>	<code>void_t a, b</code>	Returns 0 if equal (always), else 1. Static.
<code>sort</code>	<code>void</code>	<code>ref void_t vec[\$]</code>	No-op sort. Static.
<code>check_trait</code>	<code>bit</code>	<code>bit trait, bit exp, int line, string file</code>	Assert <code>trait==exp</code> ; call <code>fail()</code> if not. Static.
<code>fail</code>	<code>void</code>	<code>int line, string file</code>	Issue \$fatal with optional file/line. Static.

`object_traits`

class `extends` `void_traits`

Traits for object-derived classes. Delegates `equal()/compare()` to `object::compare()`.

Method	Return	Args	Description
<code>equal</code>	<code>bit</code>	<code>object a, b</code>	True if <code>a.compare(b)==0</code> . Static.
<code>compare</code>	<code>int32_t</code>	<code>object a, b</code>	Returns <code>a.compare(b)</code> . Static.

Method	Return	Args	Description
sort	void	ref object vec[\$]	No-op (objects are unordered). Static.

`class_traits#(T)`

class **extends** void_traits

Traits for arbitrary class handles. empty is null. Uses identity (==) for equality.

Method	Return	Args	Description
equal	bit	T a, T b	True if a==b (handle identity). Static.
compare	int32_t	T a, T b	Returns 0 if equal, else 1. Static.
sort	void	ref T vec[\$]	No-op. Static.

`base_int_traits#(T)`

class **extends** void_traits

Parameterized base traits for integral scalar types. empty=T'(0). Provides arithmetic compare and built-in sort.

Method	Return	Args	Description
equal	bit	T a, T b	True if a==b. Static.
compare	int32_t	T a, T b	Returns 1 if a>b, -1 if a<b, 0 if equal. Static.
sort	void	ref T vec[\$]	Sort using SystemVerilog built-in queue sort. Static.

`base_unsigned_int_traits#(T)`

class **extends** base_int_traits#(T)

Sets is_signed=0, is_unsigned=1. Convenience base for all unsigned integer traits typedefs.

`base_signed_int_traits#(T)`

class **extends** base_int_traits#(T)

Sets is_signed=1, is_unsigned=0. Convenience base for all signed integer traits typedefs.

`base_four_state_traits#(T)`

class **extends** base_int_traits#(T)

Traits base for four-state types (logic, reg, integer, time). Sets is_two_state=0, is_four_state=1.

integer_traits

class **extends** base_four_state_traits#(integer)

Traits for the SystemVerilog integer type (four-state, signed).

reg_traits / logic_traits / time_traits

class **extends** base_four_state_traits#(reg)

Traits for reg, logic, and time (all four-state, unsigned). `logic_traits` and `time_traits` are typedefs of `reg_traits`.

bit_vector_traits#(N)

class **extends** base_int_traits#(bit[N-1:0])

Traits for packed bit-vector types of width N. `is_array=1`; `is_arithmetic=1`.

real_base_traits#(T)

class **extends** void_traits

Traits for real/shortreal. Uses epsilon-based equality (2.2e-16). Sorts elements using an item comparator.

Method	Return	Args	Description
<code>equal</code>	<code>bit</code>	<code>T a, T b</code>	True if $ a-b \leq \text{epsilon}$. Static.
<code>compare</code>	<code>int32_t</code>	<code>T a, T b</code>	1 if $a > b$ (and not equal), -1 if $a < b$, else 0. Static.
<code>sort</code>	<code>void</code>	<code>ref T vec[\$]</code>	Sort with item comparator. Static.

string_traits

class **extends** void_traits

Traits for string. `empty` is an empty string. Uses lexicographic comparison and sorting.

Method	Return	Args	Description
<code>equal</code>	<code>bit</code>	<code>string a, b</code>	True if $a == b$. Static.
<code>compare</code>	<code>int32_t</code>	<code>string a, b</code>	Lexicographic: 1 if $a > b$, -1 if $a < b$, 0 if equal. Static.
<code>sort</code>	<code>void</code>	<code>ref string vec[\$]</code>	Sort with item comparator. Static.

Containers

3.1 `object` — Base Object

File: `src/containers/object.svh`

Defines `void_t` (the empty base class, used as the "void" type throughout SVX) and `object` (adds virtual `to_str`, `compare`, and `copy` to `void_t`). All containers and tree nodes derive ultimately from `object`.

`void_t`

class

The empty class. Serves as the null base type and as the type of the "empty" element for `void_traits`.

`object`

class **extends** `void_t`

Base object providing virtual `to_str()`, `compare()`, and `copy()`.

Method	Return	Args	Description
<code>to_str</code>	<code>string</code>	—	Return a human-readable string (default: "").
<code>compare</code>	<code>int</code>	<code>object obj</code>	Compare to <code>obj</code> ; return 0 (default implementation).
<code>copy</code>	<code>void</code>	<code>object rhs</code>	Shallow-copy from <code>rhs</code> (default: no-op).

3.2 `container` — Container Base Classes

File: `src/containers/container.svh`

Abstract container base hierarchy. `container` defines `size/clear/is_empty`. `typed_container#(T,P)` adds a typed empty-element sentinel initialised from the traits class `P` at construction.

`container`

virtual class **extends** `void_t`

Abstract base for all containers. Defines the core `size/clear/is_empty` interface.

Method	Return	Args	Description
<code>size</code>	<code>size_t</code>	—	Return the number of elements (pure virtual).
<code>clear</code>	<code>void</code>	—	Remove all elements (pure virtual).
<code>is_empty</code>	<code>bit</code>	—	True if container holds no elements (pure virtual).

`typed_container#(T, P)`

virtual class **extends** `container`

Typed abstract base. Provides a typed empty-element sentinel initialised from the traits class P at construction.

Constructor: `new(--)` — Initialise the typed empty-element sentinel from the traits class P.

Method	Return	Args	Description
<code>size</code>	<code>size_t</code>	—	Overridable; returns 0 by default.
<code>is_empty</code>	<code>bit</code>	—	Overridable; returns 0 by default.
<code>clear</code>	<code>void</code>	—	Overridable; no-op by default.

3.3 vector

File: src/containers/vector.svh

Dynamic array backed by a SystemVerilog queue. Provides random read/write with automatic extension, shallow copy/clone, equality comparison, sort, and bulk append operations. The base class for queue, stack, and deque.

`vector#(T, P)`

class **extends** `typed_container#(T,P)`

Ordered, random-access dynamic array.

Method	Return	Args	Description
<code>create</code>	<code>this_t</code>	<code>vector_type list</code>	Static factory: create a vector preloaded from a queue literal.
<code>size</code>	<code>size_t</code>	—	Number of elements currently in the vector.
<code>is_empty</code>	<code>bit</code>	—	True when <code>size()==0</code> .
<code>clear</code>	<code>void</code>	—	Delete all elements; reset to empty state.
<code>extend</code>	<code>void</code>	<code>size_t sz</code>	Push <code>sz</code> empty elements, growing the vector by that amount.

Method	Return	Args	Description
<code>write</code>	<code>void</code>	<code>index_t idx</code> , <code>T t</code>	Write element at <code>idx</code> ; auto-extends if <code>idx >= size()</code> .
<code>read</code>	<code>T</code>	<code>index_t idx</code>	Read element at <code>idx</code> ; returns the empty sentinel if out of range.
<code>copy</code>	<code>void</code>	<code>this_t vec</code>	Shallow copy from <code>vec</code> into this vector.
<code>clone</code>	<code>this_t</code>	—	Return a new vector that is a shallow copy of this one.
<code>compare</code>	<code>int</code>	<code>this_t v</code>	Returns 0 if equal to <code>v</code> , else non-zero.
<code>equal</code>	<code>bit</code>	<code>this_t v</code>	Element-wise equality using <code>P::equal()</code> .
<code>sort</code>	<code>void</code>	—	Sort elements using <code>P::sort()</code> .
<code>appendv</code>	<code>void</code>	<code>vector_type v</code>	Append a queue literal to this vector.
<code>appendc</code>	<code>void</code>	<code>T t</code>	Append a single element to this vector.
<code>append</code>	<code>void</code>	<code>this_t v</code>	Append all elements of vector <code>v</code> to this vector.

3.4 map

File: `src/containers/map.svh`

Class-based dynamic associative array. Allows sparse key-value storage allocated on demand and passed by reference. `singleton_map` provides a global instance accessible via `get_inst()`.

`map#(KEY, T, P)`

class `extends` `typed_container#(T,P)`

Associative array keyed by `KEY`, storing values of type `T`.

Method	Return	Args	Description
<code>get</code>	<code>T</code>	<code>KEY key</code>	Return item for key; returns the empty sentinel if key not found.
<code>insert</code>	<code>bit</code>	<code>KEY key</code> , <code>T item</code>	Insert (key,item). Returns 1 if key is new, 0 if duplicate (value overwritten).
<code>size</code>	<code>size_t</code>	—	Number of key-value pairs.
<code>is_empty</code>	<code>bit</code>	—	True when <code>size()==0</code> .
<code>delete</code>	<code>bit</code>	<code>KEY key</code>	Remove entry for key. Returns 1 on success, 0 if not found.
<code>clear</code>	<code>void</code>	—	Remove all entries.
<code>exists</code>	<code>bit</code>	<code>KEY key</code>	True if key exists in the map.
<code>copy</code>	<code>void</code>	<code>this_t rhs</code>	Shallow copy from <code>rhs</code> into this map.
<code>clone</code>	<code>this_t</code>	—	Return a new map that is a shallow copy of this one.

Method	Return	Args	Description
<code>compare</code>	<code>int32_t</code>	<code>this_t m</code>	Returns 0 if maps are equal, else non-zero.
<code>equal</code>	<code>bit</code>	<code>this_t m</code>	True if all keys and <code>P::equal()</code> -checked values match.
<code>first</code>	<code>bit</code>	<code>ref KEY index</code>	Move to first key (smallest); returns 0 if empty. For iterator use.
<code>last</code>	<code>bit</code>	<code>ref KEY index</code>	Move to last key (largest). For iterator use.
<code>next</code>	<code>bit</code>	<code>ref KEY index</code>	Advance to next key. For iterator use.
<code>prev</code>	<code>bit</code>	<code>ref KEY index</code>	Move to previous key. For iterator use.

`singleton_map#(KEY, T, P)`

class `extends map#(KEY,T,P)`

A map of which only one instance exists per type parameterization.

Method	Return	Args	Description
<code>get_inst</code>	<code>this_t</code>	—	Return the global singleton instance, creating it on the first call. Static.

3.5 queue

File: src/containers/queue.svh

FIFO queue derived from vector. `put()` inserts at the head; `get()` removes from the tail. `fixed_size_queue` adds a maximum-size constraint and a push-success status.

`queue#(T, P)`

class `extends vector#(T,P)`

FIFO queue: items enter at the front and leave from the back.

Method	Return	Args	Description
<code>put</code>	<code>void</code>	<code>T t</code>	Push item onto the head (front) of the queue.
<code>get</code>	<code>T</code>	—	Pop and return item from the tail; returns empty if queue is empty.
<code>peek</code>	<code>T</code>	—	Return the tail item without removing it.
<code>is_empty</code>	<code>bit</code>	—	True when <code>size()==0</code> .
<code>clone</code>	<code>this_t</code>	—	Return a shallow copy of this queue.

`fixed_size_queue#(T, P)`

class `extends` `queue#(T,P)`

Queue that rejects pushes when the maximum size is reached.

Constructor: `new(size_t n=1)` — Set maximum capacity to `n` (minimum 1).

Method	Return	Args	Description
<code>set_max_size</code>	<code>void</code>	<code>size_t n</code>	Update the capacity limit; clamps to current size if <code>n</code> is too small.
<code>get_max_size</code>	<code>size_t</code>	—	Return the current capacity limit.
<code>put</code>	<code>void</code>	<code>T t</code>	Push item; silently rejected if at capacity.
<code>last_push_succeeded</code>	<code>bit</code>	—	True if the most recent <code>put()</code> succeeded.
<code>is_full</code>	<code>bit</code>	—	True when <code>size() >= max_size</code> .
<code>copy</code>	<code>void</code>	<code>this_t q</code>	Copy contents and capacity from <code>q</code> .
<code>clone</code>	<code>this_t</code>	—	Return a shallow copy.
<code>equal</code>	<code>bit</code>	<code>vector#() v</code>	True if element contents and capacity match.

3.6 stack

File: src/containers/stack.svh

LIFO stack derived from `vector`. `push()` and `pop()` operate on the front (index 0).

`stack#(T, P)`

class `extends` `vector#(T,P)`

LIFO stack: top of stack is index 0.

Method	Return	Args	Description
<code>push</code>	<code>void</code>	<code>T t</code>	Push item onto the top of the stack.
<code>pop</code>	<code>T</code>	—	Remove and return the top item; returns empty if stack is empty.
<code>peek</code>	<code>T</code>	—	Return top item without removing it.
<code>is_empty</code>	<code>bit</code>	—	True when <code>size()==0</code> .
<code>clone</code>	<code>this_t</code>	—	Return a shallow copy of this stack.

3.7 deque

File: src/containers/deque.svh

Double-ended queue derived from `vector`. Supports `push/pop` at both ends, `shuffle`, and `reverse`.

`deque#(T, P)`

class **extends** `vector#(T,P)`

Double-ended queue with push/pop at both front and back.

Method	Return	Args	Description
<code>push_front</code>	<code>void</code>	<code>T t</code>	Insert item at the front.
<code>push_back</code>	<code>void</code>	<code>T t</code>	Insert item at the back.
<code>pop_front</code>	<code>T</code>	—	Remove and return front item.
<code>pop_back</code>	<code>T</code>	—	Remove and return back item.
<code>shuffle</code>	<code>void</code>	—	Randomly reorder elements.
<code>reverse</code>	<code>void</code>	—	Reverse the order of elements.
<code>clone</code>	<code>this_t</code>	—	Return a shallow copy of this deque.

3.8 `circ_buf` — Circular Buffer

File: `src/containers/circ_buf.svh`

Fixed-capacity circular (ring) buffer backed by a static array of size `S`, parameterised on element type `T`, traits `P`, and compile-time capacity `S`. Operates in one of two modes. In the default reject mode a push into a full buffer is refused with an error message. In overwrite mode a push into a full buffer silently overwrites the oldest item at the tail.

`circ_buf#(T, P, S)`

class **extends** `typed_container#(T,P)`

Fixed-size circular buffer holding `S` elements in FIFO ordering.

Constructor: `new(--)` — Calls `clear()` to initialise head, tail, and count.

Method	Return	Args	Description
<code>size</code>	<code>size_t</code>	—	Number of items currently in the buffer.
<code>is_empty</code>	<code>bit</code>	—	True when <code>count==0</code> .
<code>is_full</code>	<code>bit</code>	—	True when <code>count==S</code> .
<code>clear</code>	<code>void</code>	—	Reset head, tail, and count to zero.
<code>push</code>	<code>void</code>	<code>T t</code> , <code>circ_buf_mode_e</code>	Insert <code>t</code> ; in <code>REJECT_MODE</code> prints error if full. In <code>OVERWRITE_MODE</code> the tail item is overwritten.
<code>pop</code>	<code>T</code>	—	Remove and return head item; prints error and returns empty if buffer is empty.

3.9 pair / triple / quadruple

File: src/containers/pair.svh

Lightweight tuple containers for binding two, three, or four values of potentially different types. All extend `pair#(T1,T2)` and derive from `void_t` (use `void_traits` when placing in a container).

`pair#(T1, T2)`

class `extends` `void_t`

Holds two values of types T1 and T2.

Constructor: `new(T1 a, T2 b)` — Initialise both fields.

Method	Return	Args	Description
<code>first</code>	T1	—	Return the first element.
<code>second</code>	T2	—	Return the second element.
<code>set_first</code>	void	T1 <code>t</code>	Replace the first element.
<code>set_second</code>	void	T2 <code>t</code>	Replace the second element.

`triple#(T1, T2, T3)`

class `extends` `pair#(T1,T2)`

Extends pair with a third element of type T3.

Constructor: `new(T1 a, T2 b, T3 c)` — Initialise all three fields.

Method	Return	Args	Description
<code>third</code>	T3	—	Return the third element.
<code>set_third</code>	void	T3 <code>t</code>	Replace the third element.

`quadruple#(T1, T2, T3, T4)`

class `extends` `triple#(T1,T2,T3)`

Extends triple with a fourth element of type T4.

Constructor: `new(T1 a, T2 b, T3 c, T4 d)` — Initialise all four fields.

Method	Return	Args	Description
<code>fourth</code>	T4	—	Return the fourth element.
<code>set_fourth</code>	void	T4 <code>t</code>	Replace the fourth element.

3.10 sorter

File: src/containers/sorter.svh

Generic sort implementation for SystemVerilog queues. The comparator is provided by the traits parameter P.

`sorter#(T, P)`

class

Sorts queues of type T using P::compare() as the element comparator.

Method	Return	Args	Description
<code>sort</code>	<code>void</code>	<code>ref T vec[\$]</code>	Public entry point: sort vec in place. Static.
<code>qsort</code>	<code>void</code>	<code>ref T vec[\$],</code> <code>int32_t l,</code> <code>int32_t r</code>	Sort the subrange [l, r] of vec recursively. Static.

Iterators

4.1 Iterator Interfaces

File: src/iterators/iterator_intf.svh

The iterator interface hierarchy. `iterator_intf_base` defines `set/get/skip/size/is_empty`. `fwd_intf` adds forward traversal (`first/next/is_last/at_end`). `bkwd_intf` adds backward traversal (`last/prev/is_first/at_beginning`). `random_intf` adds random selection.

`iterator_intf_base#(T, P)`

interface class

Base iterator interface. All iterators implement this.

Method	Return	Args	Description
<code>set</code>	<code>void</code>	<code>T t</code>	Write value T at the current position (pure virtual).
<code>get</code>	<code>T</code>	—	Read the value at the current position (pure virtual).
<code>skip</code>	<code>bit</code>	<code>signed_index_t d</code>	Move forward (<code>d>0</code>) or backward (<code>d<0</code>) by d steps (pure virtual).
<code>size</code>	<code>size_t</code>	—	Size of the bound container (pure virtual).
<code>is_empty</code>	<code>bit</code>	—	True if the iterator is unbound or the container is empty (pure virtual).

`fwd_intf#(T, P)`

interface class `extends` `iterator_intf_base#(T,P)`

Forward-traversal interface.

Method	Return	Args	Description
<code>first</code>	<code>bit</code>	—	Reset to first element; returns 0 if empty (pure virtual).
<code>next</code>	<code>bit</code>	—	Advance one step; may enter end state (pure virtual).
<code>is_last</code>	<code>bit</code>	—	True if pointing at the last element (pure virtual).

Method	Return	Args	Description
<code>at_end</code>	<code>bit</code>	—	True if past the last element (pure virtual).

`bkwd_intf#(T, P)`

interface class **extends** `iterator_intf_base#(T,P)`

Backward-traversal interface.

Method	Return	Args	Description
<code>last</code>	<code>bit</code>	—	Reset to last element; returns 0 if empty (pure virtual).
<code>prev</code>	<code>bit</code>	—	Move one step backward (pure virtual).
<code>is_first</code>	<code>bit</code>	—	True if pointing at the first element (pure virtual).
<code>at_beginning</code>	<code>bit</code>	—	True if before the first element (pure virtual).

`random_intf#(T, P)`

interface class **extends** `iterator_intf_base#(T,P)`

Random-access iterator interface.

Method	Return	Args	Description
<code>random</code>	<code>bit</code>	—	Select a random valid element (pure virtual).
<code>set_seed</code>	<code>void</code>	<code>int seed</code>	Set the RNG seed (pure virtual).
<code>set_default_seed</code>	<code>void</code>	—	Restore the default RNG seed (pure virtual).

4.2 List Iterators

File: `src/iterators/list_iterators.svh`

Iterators for `vector#()` and any class derived from it, including `stack`, `queue`, and `deque`. `list_iterator_base` provides the common binding and `set/get/skip/size/is_empty` logic. `list_iterator` traverses the container in either the forward or backward direction. `list_random_iterator` chooses a random item.

`list_iterator_base#(T, P)`

virtual class

Abstract base for list iterators. Manages the binding to a vector and provides the `set/get/skip/size/is_empty` operations shared by all list iterators.

Constructor: `new(list_t list=null)` — Optionally bind a vector.

Method	Return	Args	Description
<code>bind_list</code>	<code>void</code>	<code>list_t list</code>	Bind iterator to a vector (or unbind if null).
<code>set</code>	<code>void</code>	<code>T t</code>	Write T at the current index position.
<code>get</code>	<code>T</code>	—	Read value at the current index position.
<code>skip</code>	<code>bit</code>	<code>signed_index_t d</code>	Move index by d; clamps to valid range.
<code>size</code>	<code>size_t</code>	—	Return size of the bound vector.
<code>is_empty</code>	<code>bit</code>	—	True if unbound or vector is empty.

`list_iterator#(T, P)`

class `extends` `list_iterator_base#(T,P)` **implements** `fwd_intf#(T,P)`, `bkwd_intf#(T,P)`

Traverse a vector-derived container in either the forward or backward direction.

Constructor: `new(list_t list=null)` — Optionally bind a vector.

Method	Return	Args	Description
<code>first</code>	<code>bit</code>	—	Set index to 0; returns 0 if empty.
<code>next</code>	<code>bit</code>	—	Increment index; returns 0 if past end.
<code>is_last</code>	<code>bit</code>	—	True if index is at the last valid element.
<code>at_end</code>	<code>bit</code>	—	True if index $\geq$ size.
<code>last</code>	<code>bit</code>	—	Set index to last element; returns 0 if empty.
<code>prev</code>	<code>bit</code>	—	Decrement index; returns 0 if already at start.
<code>is_first</code>	<code>bit</code>	—	True if index == 0.
<code>at_beginning</code>	<code>bit</code>	—	True if index < 0 (before first element).

`list_random_iterator#(T, P)`

class `extends` `list_iterator_base#(T,P)` **implements** `random_intf#(T,P)`

Choose a random item in the bound vector.

Constructor: `new(list_t list=null)` — Optionally bind a vector; sets default RNG seed.

Method	Return	Args	Description
<code>random</code>	<code>bit</code>	—	Uniformly choose a random index in $[0, \text{size}-1]$.

Method	Return	Args	Description
<code>set_seed</code>	void	int seed	Seed the RNG via <code>\$urandom(seed)</code> .
<code>set_default_seed</code>	void	—	Reset to the default seed (1).

4.3 Map Iterators

File: `src/iterators/map_iterators.svh`

Iterators for the map container. `map_iterator_base` provides a means for binding a map to an iterator and for setting and getting the item at the current position. `map_iterator` traverses the map in either the forward or backward direction. `map_random_iterator` provides random selection. Map keys are traversed in the natural sorted order of the underlying SV associative array.

`map_iterator_base#(KEY, T, P)`

virtual class

Abstract base for all map iterators. Provides a means for binding a map to the iterator and for setting and getting the item at the current iterator position.

Constructor: `new(map_t map=null)` — Optionally bind a map.

Method	Return	Args	Description
<code>bind_map</code>	void	map_t m	Bind iterator to map m.
<code>get_index</code>	KEY	—	Return the current key.
<code>size</code>	size_t	—	Return number of entries in the bound map.
<code>is_empty</code>	bit	—	True if unbound or map is empty.

`map_iterator#(KEY, T, P)`

class **extends** `map_iterator_base#(KEY,T,P)` **implements** `fwd_intf#(T,P)`, `bkwd_intf#(T,P)`

Traverse a map in either the forward or backward direction. In addition to the forward and backward traversal methods, provides `skip()`, which can move in either direction.

Constructor: `new(map_t map=null)` — Optionally bind a map.

Method	Return	Args	Description
<code>set</code>	void	T t	Update the value at the current key.
<code>get</code>	T	—	Return the value at the current key.
<code>get_index</code>	KEY	—	Return the current key.
<code>first</code>	bit	—	Move to the first (smallest) key; returns 0 if the map is empty.
<code>next</code>	bit	—	Advance to the next key; returns 0 if at the end of the map.

Method	Return	Args	Description
<code>is_last</code>	<code>bit</code>	—	True if current key equals the last key in the map.
<code>at_end</code>	<code>bit</code>	—	True if the iterator is past the last entry in the map.
<code>last</code>	<code>bit</code>	—	Move to the last (largest) key.
<code>prev</code>	<code>bit</code>	—	Move to the previous key; returns 0 if already at the first entry.
<code>is_first</code>	<code>bit</code>	—	True if current key equals first key in the map.
<code>at_beginning</code>	<code>bit</code>	—	True if the iterator is before the first entry in the map.
<code>skip</code>	<code>bit</code>	<code>signed_index_t d</code>	Skip <code>d</code> steps via repeated <code>next()/prev()</code> calls.

`map_random_iterator#(KEY, T, P)`

class `extends` `map_iterator_base#(KEY,T,P)` `implements` `random_intf#(T,P)`

Randomly selects a map entry.

Constructor: `new(map_t map=null)` — Optionally bind a map; set default RNG seed.

Method	Return	Args	Description
<code>set</code>	<code>void</code>	<code>T t</code>	Update value at current key.
<code>get</code>	<code>T</code>	—	Return value at current key.
<code>random</code>	<code>bit</code>	—	Select a uniformly random entry.
<code>set_seed</code>	<code>void</code>	<code>int seed</code>	Seed the RNG via <code>\$urandom(seed)</code> .
<code>set_default_seed</code>	<code>void</code>	—	Reset to default seed (1).

4.4 Permutation Iterators

File: `src/iterators/permute_iterators.suh`

Iterators that generate all permutations of a bound vector without modifying it. An internal index ordering tracks the current permutation; the bound vector itself is never changed. The original ordering of the bound vector is permutation 0. A vector of length n has $n!$ permutations. `get_nth(n)` returns the n th element of the current permutation; `get_permutation_vector()` returns the current element ordering. `permute_iterator` traverses all permutations sequentially in both directions; `permute_random_iterator` jumps to a randomly chosen permutation. Once a vector is bound, it should not be modified; call `initialize()` to reset if needed.

`permute_iterator_base#(T, P)`

class

Shared permutation state and algorithm. Maintains the current permutation ordering and the total permutation count, both computed and cached at bind time.

Constructor: `new(vec_t v=null)` — Optionally bind a vector and initialise the permutation state.

Method	Return	Args	Description
<code>bind_vector</code>	<code>void</code>	<code>vec_t v</code>	Bind a vector; calls <code>initialize()</code> .
<code>size</code>	<code>size_t</code>	—	Size of the bound vector.
<code>is_empty</code>	<code>bit</code>	—	True if unbound or empty.
<code>get_nth</code>	<code>T</code>	<code>index_t n</code>	Return the <i>n</i> th element for the current permutation.
<code>get_permutation_index</code>	<code>longint</code>	—	Return the current permutation index (0-based).
<code>get_permutation_vector</code>	<code>vector#()</code>	—	Return the current permutation ordering as a vector of element indices.
<code>skip</code>	<code>bit</code>	<code>signed_index_t d</code>	Advance (positive <i>d</i>) or retreat (negative <i>d</i>) by <i>d</i> permutations.
<code>print</code>	<code>void</code>	—	Display the current permutation index and element ordering to <code>stdout</code> .

`permute_iterator#(T, P)`

class `extends` `permute_iterator_base#(T,P)` `implements` `fwd_intf#(T,P)`, `bkwd_intf#(T,P)`

Sequential bidirectional traversal of all permutations.

Constructor: `new(vec_t v=null)` — Optionally bind a vector.

Method	Return	Args	Description
<code>first</code>	<code>bit</code>	—	Reset to permutation 0.
<code>next</code>	<code>bit</code>	—	Advance to next permutation; returns 0 if already at last.
<code>is_last</code>	<code>bit</code>	—	True if at the last permutation.
<code>at_end</code>	<code>bit</code>	—	True if <code>pix >= max_pix</code> .
<code>last</code>	<code>bit</code>	—	Jump to the last permutation.
<code>prev</code>	<code>bit</code>	—	Return to the previous permutation.
<code>is_first</code>	<code>bit</code>	—	True if <code>pix==0</code> .
<code>at_beginning</code>	<code>bit</code>	—	True if <code>pix<0</code> .

`permute_random_iterator#(T, P)`

class `extends` `permute_iterator_base#(T,P)` `implements` `random_intf#(T,P)`

Jump to a randomly chosen permutation.

Constructor: `new(vec_t v=null)` — Optionally bind a vector; set default RNG seed.

Method	Return	Args	Description
<code>random</code>	<code>bit</code>	—	Select a uniformly random permutation index.
<code>set_seed</code>	<code>void</code>	<code>int seed</code>	Seed the RNG via <code>\$urandom(seed)</code> .

Method	Return	Args	Description
set_default_seed	void	—	Reset to default seed (1).

4.5 range

File: src/iterators/range.svh

A view over a contiguous sub-range [lb, ub] of an existing iterator. `range_base` stores and validates the bounds. `range#(T,P)` wraps a `fwd_intf` iterator and optionally a `bkwd_intf` if the underlying iterator supports it; backward access is silently unavailable otherwise.

range_base

virtual class

Abstract base that validates and stores index bounds; corrects inverted bounds automatically.

Constructor: `new(size_t size, index_t lb, index_t ub)` — Initialise the bounds, clamping to the container size and correcting any inversion.

Method	Return	Args	Description
get_lower_bound	index_t	—	Return lb.
get_upper_bound	index_t	—	Return ub.

range#(T, P)

class **extends** `range_base` **implements** `fwd_intf#(T,P)`, `bkwd_intf#(T,P)`

Sub-range iterator. Binds to a `fwd_intf`; optionally acquires backward capability via `$cast`.

Constructor: `new(iterator_intf_base#() it, index_t lb, ub)` — Bind to iterator `it` with the given range bounds; fails fatally if it does not support forward traversal.

Method	Return	Args	Description
set	void	T t	Set value at current iterator position.
get	T	—	Get value at current iterator position.
size	size_t	—	Number of elements in the range: <code>ub-lb+1</code> .
is_empty	bit	—	True if underlying iterator is null or empty.
first	bit	—	Move underlying iterator to position lb.
next	bit	—	Advance one step within range bounds.
is_last	bit	—	True if current position $\geq$ ub.
at_end	bit	—	True if position $>$ ub.

Method	Return	Args	Description
<code>last</code>	<code>bit</code>	—	Move to position <code>ub</code> (requires <code>bkwd_intf</code>).
<code>prev</code>	<code>bit</code>	—	Move one step backward (requires <code>bkwd_intf</code>).
<code>is_first</code>	<code>bit</code>	—	True if current position == <code>lb</code> .
<code>at_beginning</code>	<code>bit</code>	—	True if position < <code>lb</code> .
<code>skip</code>	<code>bit</code>	<code>signed_index_t</code> <code>d</code>	Delegate to underlying <code>fwd_iter.skip(d)</code> .

4.6 `map_range`

File: `src/iterators/map_range.svh`

Extends `range` for map iterators, adding `get_index()` to retrieve the current map key from within a sub-range view.

`map_range#(KEY, T, P)`

class `extends range#(T,P)`

Range view over a map iterator slice, exposing the current key.

Constructor: `new(map_iter_t it, index_t lb, ub)` — Delegate to `range`; retain reference to the map iterator.

Method	Return	Args	Description
<code>get_index</code>	<code>KEY</code>	—	Return the key at the current iterator position.

Algorithms

5.1 algo — Core Algorithms

File: *src/algorithms/algo.svh*

Generic single-iterator algorithms. All functions are static and accept a `fwd_intf` iterator plus, where needed, a `predicate` or `fcn` object. `find()` positions the iterator at the first matching element rather than returning it.

`algo#(T, P)`

class

Collection of generic single-iterator algorithms.

Method	Return	Args	Description
<code>count</code>	<code>uint32_t</code>	<code>fwd_intf#()</code> <code>iter,</code> <code>predicate#(T)</code> <code>p</code>	Count elements for which <code>p.is_true()</code> . Static.
<code>all_of</code>	<code>bit</code>	<code>fwd_intf#()</code> <code>iter,</code> <code>predicate#(T)</code> <code>p</code>	True if predicate holds for every element. Static.
<code>none_of</code>	<code>bit</code>	<code>fwd_intf#()</code> <code>iter,</code> <code>predicate#(T)</code> <code>p</code>	True if predicate holds for no element. Static.
<code>any_of</code>	<code>bit</code>	<code>fwd_intf#()</code> <code>iter,</code> <code>predicate#(T)</code> <code>p</code>	True if predicate holds for at least one element. Static.
<code>min</code>	<code>T</code>	<code>fwd_intf#()</code> <code>iter</code>	Return minimum element using <code>P::compare()</code> . Static.
<code>max</code>	<code>T</code>	<code>fwd_intf#()</code> <code>iter</code>	Return maximum element using <code>P::compare()</code> . Static.
<code>find</code>	<code>void</code>	<code>fwd_intf#()</code> <code>iter,</code> <code>predicate#(T)</code> <code>p</code>	Position <code>iter</code> at first element where <code>p.is_true()</code> ; no-op if not found. Static.

Method	Return	Args	Description
<code>for_each</code>	<code>void</code>	<code>fwd_intf#()</code> <code>iter, fcn#(T)</code> <code>fn</code>	Call <code>fn.f(element)</code> for every element. Static.

5.2 algo2 — Dual-Iterator Algorithms

File: *src/algorithms/algo2.svh*

Algorithms that operate on two iterators simultaneously. `zip` traverses both iterators in lock-step, calling `fn.f(t1, t2)` at each step until either iterator reaches its end.

`algo2#(T1, P1, T2, P2)`

class

Algorithms for paired traversal of two differently-typed iterators.

Method	Return	Args	Description
<code>zip</code>	<code>void</code>	<code>fwd_intf#(T1,P1)</code> <code>iter1,</code> <code>fwd_intf#(T2,P2)</code> <code>iter2,</code> <code>fcn2#(T1,T2)</code> <code>fn</code>	Traverse <code>iter1</code> and <code>iter2</code> in lock-step; call <code>fn.f(t1,t2)</code> at each step. Stops when either iterator ends. Static.

5.3 accum — Accumulation Algorithms

File: *src/algorithms/accum.svh*

Algorithms that carry a mutable accumulator `A` across iterations. `accum` operates on one iterator; `accum2` operates on two in lock-step.

`accum#(T, P, A)`

class

Single-iterator accumulation algorithm.

Method	Return	Args	Description
<code>accumulate</code>	<code>void</code>	<code>fwd_intf#()</code> <code>iter,</code> <code>accum_fcn#(T,A)</code> <code>fn, ref A acc</code>	Call <code>fn.f(element, acc)</code> for every element; <code>acc</code> is updated in place by <code>fn</code> . Static.

`accum2#(T1, P1, T2, P2, A)`

class

Dual-iterator accumulation algorithm.

Method	Return	Args	Description
<code>accumulate</code>	<code>void</code>	<code>fwd_intf#(T1,P1)</code> <code>iter1,</code> <code>fwd_intf#(T2,P2)</code> <code>iter2,</code> <code>accum_fcn2#()</code> <code>fn, ref A acc</code>	Traverse both iterators in lock-step; call <code>fn.f(t1, t2, acc)</code> at each step. Static.

5.4 fcn — Function Objects

File: src/algorithms/fcn.svh

Virtual function objects (SVX's substitute for C++ functors and lambdas). Users derive concrete classes and override `f()`. The hierarchy provides one-argument, two-argument, and accumulating variants.

`fcn_base`

virtual class

Empty base class for all function objects.

`fcn#(T)`

virtual class **extends** `fcn_base`

Abstract function object taking one argument of type `T`.

Method	Return	Args	Description
<code>f</code>	<code>void</code>	<code>T t</code>	Apply the function to <code>t</code> (pure virtual).

`fcn2#(T1, T2)`

virtual class **extends** `fcn_base`

Abstract function object taking two arguments of types `T1` and `T2`.

Method	Return	Args	Description
<code>f</code>	<code>void</code>	<code>T1 t1, T2 t2</code>	Apply the function to <code>(t1, t2)</code> (pure virtual).

`accum_fcn#(T, A)`

virtual class **extends** `fcn_base`

Abstract function object taking one value and a mutable accumulator.

Method	Return	Args	Description
<code>f</code>	<code>void</code>	<code>T t, ref A a</code>	Apply the function; update accumulator <code>a</code> in place (pure virtual).

`accum_fcn2#(T1, T2, A)`

virtual class **extends** `fcn_base`

Abstract function object taking two values and a mutable accumulator.

Method	Return	Args	Description
<code>f</code>	<code>void</code>	<code>T1 t1, T2 t2, ref A a</code>	Apply the function to <code>(t1, t2)</code> ; update <code>a</code> in place (pure virtual).

5.5 predicate — Predicate Classes

File: src/algorithms/predicate.svh

Virtual predicate hierarchy. A predicate is a Boolean function over a single element `T`. Concrete predicate classes are composed with `and_pred`, `or_pred`, and `not_pred`.

`predicate_base`

virtual class

Empty base class for all predicate classes.

`predicate#(T)`

virtual class **extends** `predicate_base`

Abstract predicate: `is_true()` is pure virtual; `is_false()` delegates.

Method	Return	Args	Description
<code>is_true</code>	<code>bit</code>	<code>T t</code>	Return true if <code>t</code> satisfies the predicate (pure virtual).
<code>is_false</code>	<code>bit</code>	<code>T t</code>	Return <code>!is_true(t)</code> .

`and_pred#(T)`

class **extends** `predicate#(T)`

True if both sub-predicates `a` and `b` are true.

Constructor: `new(predicate#(T) _a, _b)` — Store references to a and b.

Method	Return	Args	Description
<code>is_true</code>	<code>bit</code>	<code>T t</code>	Return <code>a.is_true(t) && b.is_true(t)</code> .

`or_pred#(T)`

class `extends` `predicate#(T)`

True if at least one of the two sub-predicates is true.

Constructor: `new(predicate#(T) _a, _b)` — Store references to a and b.

Method	Return	Args	Description
<code>is_true</code>	<code>bit</code>	<code>T t</code>	Return <code>a.is_true(t) b.is_true(t)</code> .

`not_pred#(T)`

class `extends` `predicate#(T)`

True when the wrapped predicate is false.

Constructor: `new(predicate#(T) _p)` — Store reference to p.

Method	Return	Args	Description
<code>is_true</code>	<code>bit</code>	<code>T t</code>	Return <code>p.is_false(t)</code> .

`always_true#(T)`

class `extends` `predicate#(T)`

Constant predicate that always evaluates to 1.

Method	Return	Args	Description
<code>is_true</code>	<code>bit</code>	<code>T t</code>	Always returns 1.

`always_false#(T)`

class `extends` `predicate#(T)`

Constant predicate that always evaluates to 0.

Method	Return	Args	Description
<code>is_true</code>	<code>bit</code>	<code>T t</code>	Always returns 0.

Linked Structures

6.1 node

File: src/linked/node.svh

A named, markable container node extending `object`. The name field supports hierarchical naming in the tree. The mark/unmark mechanism supports traversal algorithms that need to track visited nodes.

node

class **extends** `object`

Named, markable base node for tree structures.

Constructor: `new(string nm="")` — Initialise name to nm.

Method	Return	Args	Description
<code>set_name</code>	<code>void</code>	<code>string nm</code>	Set the node name.
<code>get_name</code>	<code>string</code>	—	Return the node name.
<code>mark</code>	<code>void</code>	—	Set the mark flag to 1.
<code>unmark</code>	<code>void</code>	—	Clear the mark flag to 0.
<code>get_mark</code>	<code>bit</code>	—	Return the current mark value.
<code>is_marked</code>	—	—	Return the mark flag (equivalent to <code>get_mark()</code>).

6.2 tree

File: src/linked/tree.svh

Classic tree data structure. Each node has exactly one parent (except the root, which has none), a name, and a map of named children. Each node is a typed container that can hold an object of arbitrary type. Hierarchical path names use the dot (.) as the separator; the `find()` method locates any named node by dot-separated path.

tree

class **extends** `node`

N-ary tree node with named children, full-path support, and hierarchical search.

Constructor: `new(string nm, tree p)` — Initialise the node with name nm, link to parent p, and register as a child of p.

Method	Return	Args	Description
<code>insert</code>	<code>void</code>	<code>tree t</code>	Add <code>t</code> as a named child of this node; error if name duplicate.
<code>get_parent</code>	<code>tree</code>	—	Return the parent node handle.
<code>get_full_name</code>	<code>string</code>	—	Return the fully-qualified dotted path name.
<code>first_child</code>	<code>bit</code>	<code>ref string nm</code>	Set <code>nm</code> to the name of the first child; returns 0 if none.
<code>next_child</code>	<code>bit</code>	<code>ref string nm</code>	Advance <code>nm</code> to the next sibling name; returns 0 if none.
<code>num_children</code>	<code>size_t</code>	—	Return the number of direct children.
<code>size</code>	<code>size_t</code>	—	Return the total number of nodes in this subtree (including this node), counted recursively.
<code>get_children</code>	<code>deque#()</code>	—	Return a deque of all direct child handles.
<code>get_child</code>	<code>tree</code>	<code>string nm</code>	Return the named child; null if not found.
<code>find</code>	<code>tree</code>	<code>string path</code>	Locate a node by dot-separated path. Returns null on parse error or not found.
<code>mark_all</code>	<code>void</code>	—	Recursively mark all nodes in this subtree.
<code>unmark_all</code>	<code>void</code>	—	Recursively unmark all nodes in this subtree.

6.3 tree_iterator

File: *src/linked/tree_iterator.svh*

Pre-order depth-first iterator over a tree. `size()` counts all reachable nodes from the bound root.

`tree_iterator#(T, P)`

class **implements** `fwd_intf#(T,P)`

Forward-only pre-order DFS iterator over a tree.

Constructor: `new(tree t=null)` — Bind to tree `t`.

Method	Return	Args	Description
<code>size</code>	<code>size_t</code>	—	Return the total count of nodes reachable from the bound root. Note: this call alters node marks as a side effect.
<code>is_empty</code>	<code>bit</code>	—	True if no tree is bound.
<code>set</code>	<code>void</code>	<code>tree t</code>	Has no effect; tree traversal does not support element replacement.

Method	Return	Args	Description
<code>get</code>	<code>tree</code>	—	Return the tree node at the top of the stack.
<code>first</code>	<code>bit</code>	—	Begin traversal at the root.
<code>next</code>	<code>bit</code>	—	Advance to the next node in pre-order.
<code>is_last</code>	<code>bit</code>	—	True when exactly one node remains on the stack.
<code>at_end</code>	<code>bit</code>	—	True when the stack is empty.
<code>skip</code>	<code>bit</code>	<code>signed_index_t</code> <code>d</code>	Skip <code>d</code> nodes forward (negative <code>d</code> not supported).

Set

7.1 set

File: src/set/set_impl.svh

Mathematical set. All set operations return a new set and do not mutate their operands.

set#(T, P)

class **extends** `typed_container#(T,P)`

Mathematical set with insert, membership test, and set-algebra operations.

Constructor: `new(--)` — Allocate the internal map.

Method	Return	Args	Description
<code>size</code>	<code>size_t</code>	—	Return the number of members.
<code>is_empty</code>	<code>bit</code>	—	True when the set has no members.
<code>clear</code>	<code>void</code>	—	Remove all members.
<code>insert</code>	<code>void</code>	<code>T item</code>	Add item to the set (duplicates ignored via map semantics).
<code>contains</code>	<code>bit</code>	<code>T item</code>	True if item is a member.
<code>intersection</code>	<code>set_t</code>	<code>set_t s</code>	Return a new set of elements in both this and s.
<code>set_union</code>	<code>set_t</code>	<code>set_t s</code>	Return a new set of all elements in this or s (or both).
<code>difference</code>	<code>set_t</code>	<code>set_t s</code>	Return a new set of elements in this but not in s.
<code>to_vector</code>	<code>vector#()</code>	—	Return a vector containing all set members in key order.
<code>from_vector</code>	<code>void</code>	<code>vector#() vec</code>	Insert all elements of vec into the set (skipping duplicates).

Utilities

8.1 msg — Message Interface

File: src/utils/msg.svh

A simple, uniform interface for printing diagnostic messages. All methods are static, so no instance is required. Each method prints a formatted message prefixed with its severity tag and the caller-supplied identifier.

msg

class

Static message-printing class. Wraps `$error`, `$warning`, `$info`, and `$fatal`.

Method	Return	Args	Description
<code>error</code>	<code>void</code>	<code>string id,</code> <code>string txt</code>	Print an error message via <code>\\$error</code> . Static.
<code>warning</code>	<code>void</code>	<code>string id,</code> <code>string txt</code>	Print a warning message via <code>\\$warning</code> . Static.
<code>info</code>	<code>void</code>	<code>string id,</code> <code>string txt</code>	Print an informational message via <code>\\$info</code> . Static.
<code>fatal</code>	<code>void</code>	<code>string id,</code> <code>string txt</code>	Print a fatal message and abort simulation via <code>\\$fatal(1,...)</code> . Static.

8.2 msg macros — Convenience Message Macros

File: src/utils/msg_macros.svh

Preprocessor convenience macros that forward directly to the `msg` class static functions. Using the macros avoids writing `msg::` at every call site and keeps diagnostic output consistent across the library.

msg macros

macros

Four single-statement macros, one per severity level. Each expands to the corresponding `msg` static-function call.

Method	Return	Args	Description
<code>`info_msg(id, txt)</code>	void	—	Expands <code>\texttt{msg::info(id,txt)}</code> . to
<code>`warning_msg(id, txt)</code>	void	—	Expands <code>\texttt{msg::warning(id,txt)}</code> . to
<code>`error_msg(id, txt)</code>	void	—	Expands <code>\texttt{msg::error(id,txt)}</code> . to
<code>`fatal_msg(id, txt)</code>	void	—	Expands <code>\texttt{msg::fatal(id,txt)}</code> . to

8.3 ver — Version

File: `src/utils/version.svh`

Exposes the library version number through static accessor functions. Version components are defined by macros in the library header and retrieved at run time through this class.

ver

class

Static version-query class for the SVX library.

Method	Return	Args	Description
<code>get_major_version</code>	int unsigned	—	Return the major version number. Static.
<code>get_minor_version</code>	int unsigned	—	Return the minor version number. Static.
<code>get_fix_version</code>	int unsigned	—	Return the fix/patch version number. Static.
<code>get_release_mode</code>	string	—	Return the release mode string (e.g., <code>\texttt{beta}</code>). Static.
<code>get_version_string</code>	string	—	Return the full version string. Static.
<code>print_banner</code>	void	—	Display a formatted version banner to stdout. Static.